

L12A: Logic Programming

CS1101S: Programming Methodology

Sanka Rasnayaka

November 8, 2023

Readings

- SICP JS, Sec 4.4: “Logic Programming”

Outline

- Some history
- Deductive information retrieval
- How the query system works
- Implementing the query system

Outline

- Some history
- Deductive information retrieval
- How the query system works
- Implementing the query system

History of Logic Programming

- **Early history**

- Logic programming has its foundations in mathematical logic and A.I. (Carl Hewitt: Planner, 1969)

- **Discovery**

Simultaneously, the 1970s:

- Bob Kowalski (Imperial College) discovered deduction for computation
- Alain Colmerauer (Marseille) developed Prolog, based on unification and deduction.

History of Logic Programming

- A new programming “**paradigm**”
 - Logic programming is a different way of describing computation.
- Influence

Logic programming had a deep influence on

- Database research
- Programming languages

Outline

- Some history
- Deductive information retrieval
- How the query system works
- Implementing the query system

Representing information with assertions

```
address(list("Bitdiddle", "Ben"),  
        list("Slumerville", list("Ridge", "Road"), 10))
```

```
job(list("Bitdiddle", "Ben"),  
    list("computer", "wizard"))
```

```
salary(list("Bitdiddle", "Ben"), 60000)
```

- First symbols address, job, salary are the **kind of information**
- “*arguments*” - lists, literals are ***data***

More Assertions

As resident wizard, Ben oversees the company's computer division, and he supervises two programmers (*Alyssa P. Hacker* and *Cy D. Fect*) and one technician (*Lem E. Tweakit*).
Alyssa supervises a Trainee (*Louis Reasoner*).

More information

```
can_do_job(list("computer", "wizard"),  
           list("computer", "programmer"))
```

```
can_do_job(list("computer", "wizard"),  
           list("computer", "technician"))
```

```
can_do_job(list("computer", "programmer"),  
           list("computer", "programmer", "trainee"))
```

```
can_do_job(list("administration", "assistant"),  
           list("administration", "big", "wheel"))
```

Simple queries

// Query input :

```
job ($x , list("computer", "programmer"))
```

// Query results :

```
job(list("Hacker", "Alyssa", "P"),  
    list("computer", "programmer"))
```

```
job(list("Fect", "Cy", "D"),  
    list("computer", "programmer"))
```

- **Queries** have the shape of assertions but can contain logic variables.

Another query

```
// Query input :  
address($x , $y)
```

```
// Query input :  
supervisor($x , $x)
```

- The notion of “**matching**”
- An assertion matches a query if we can **instantiate** the query’s logic variables with data and obtain the assertion.

Example matches

```
// Query input :  
job($x , list("computer", $type))  
  
// Query results :  
job(list("Bitdiddle", "Ben"),  
     list("computer", "wizard"))  
job(list("Hacker", "Alyssa", "P"),  
     list("computer", "programmer"))  
job(list("Fect", "Cy", "D"),  
     list("computer", "programmer"))  
job(list("Tweakit", "Lem", "E"),  
     list("computer", "technician"))
```

Queries with pairs

```
// Query input :  
job($x , list("computer", $type))
```

does **not** match

```
job(list("Reasoner", "Louis"),  
     list("computer", "programmer", "trainee"))
```

but

```
// Query input :  
job ($x , pair("computer", $type))
```

matches the assertion!

How? Substitute \$type with list("programmer", "trainee")

Query processing

- Find all assignments to variables in the query pattern that satisfy the pattern.
 - The kind of information specified in the pattern needs to match the kind of information in an assertion
 - The assertion must result from the pattern by instantiating the pattern variables with values
- System responds to query by listing all instantiations of the query pattern with the variable assignments that satisfy it
- Special case,

If the pattern has no variables, the query reduces to a determination of whether that pattern is in the data base. The empty assignment satisfies that pattern for that data base.

Compound Queries - and

// Query input :

```
and(job($person , list("computer", "programmer")),  
    address ($person , $where))
```

// Query results :

```
and(job(list("Hacker", "Alyssa", "P"),  
        list("computer", "programmer")),  
    address(list("Hacker", "Alyssa", "P"),  
            list("Cambridge", "Mass Ave", 78)))  
and(job(list("Fect", "Cy", "D"),  
        list("computer", "programmer")),  
    address(list("Fect", "Cy", "D"),  
            list("Cambridge", list("Ames", "Street"), 3)))
```

Matching of and queries

and $(\text{query}_1, \text{query}_2, \dots, \text{query}_n)$

is satisfied by all sets of values for the pattern variables that simultaneously satisfy

$\text{query}_1, \dots, \text{query}_n$

Compound Queries - or

```
or(supervisor ($x, list("Bitdiddle", "Ben")),  
   supervisor($x, list("Hacker", "Alyssa", "P")))
```

// matches

```
or(supervisor(list("Hacker", "Alyssa", "P"),  
              list("Bitdiddle", "Ben")),  
   supervisor(list("Hacker", "Alyssa", "P") ,  
              list("Hacker", "Alyssa" , "P")))
```

// and many more

```
or(supervisor(list("Fect", "Cy", "D"),  
              list("Bitdiddle", "Ben")),  
   supervisor(list("Fect", "Cy", "D"),  
              list("Hacker", "Alyssa", "P"))) ...
```

Matching of or queries

or ($\text{query}_1, \text{query}_2, \dots, \text{query}_n$)

is satisfied by all sets of values for the pattern variables that satisfy at least one of

$\text{query}_1, \dots, \text{query}_n$

Compound Queries - not

```
and(supervisor ($x , list("Bitdiddle", "Ben")),  
    not(job($x , list("computer", "programmer"))))
```

finds all people supervised by Ben Bitdiddle who are not computer programmers.

Matching not queries

$\text{not}(\text{query}_1)$

$\text{not}(\text{query}_1)$ is satisfied by all assignments to the pattern variables that do not satisfy query_1 .

javascript_predicate queries

javascript_predicate (predicate)

is satisfied by assignments to the pattern variables in the predicate for which the instantiated predicate is true.

```
and(salary($person, $amount),  
    javascript_predicate($amount > 30000))
```

```
// find all people whose salary  
// is greater than $30 ,000
```

New relationship - `lives_near`

How would we include a new relationship `lives_near`?

Rules: Examples

```
rule(lives_near($person_1, $person_2),  
    and(address($person_1, pair($town, $rest_1)),  
        address($person_2, pair($town, $rest_2)),  
        not(same($person_1, $person_2))))
```

```
rule(same($x , $x))
```

```
rule(wheel($person),  
    and(supervisor($middle_manager, $person),  
        supervisor($x , $middle_manager)))
```

The shape of rules

General form of a rule

`rule(conclusion, body)`

where conclusion is a pattern and body is any query.

Meaning of rules

A rule represents a large (even infinite) set of assertions:

All instantiations of the rule conclusion with variable assignments that satisfy the rule body.

Rules example

```
rule(lives_near($person_1, $person_2),  
    and(address($person_1, pair($town, $rest_1)),  
        address($person_2, pair($town, $rest_2)),  
        not(same($person_1, $person_2))))
```

// Query input :

```
lives_near ($x, list("Bitdiddle", "Ben" ))
```

// Query results :

```
lives_near(list("Reasoner", "Louis"),  
           list("Bitdiddle", "Ben"))  
lives_near(list("Aull", "DeWitt"),  
           list("Bitdiddle", "Ben"))
```

Rules can be used in compound queries

```
and(job($x , pair("computer", $something)),  
    lives_near($x , list ("Bitdiddle" , "Ben")))
```

Recursive rules

```
rule(outranked_by($staff_person, $boss),  
    or(supervisor($staff_person, $boss),  
        and(supervisor($staff_person, $middle_manager),  
            outranked_by($middle_manager, $boss))))
```

Appending lists in functional programming

```
function append(x, y){  
    return is_null(x)  
        ? y  
        : pair(head(x), append(tail(x), y));  
}
```

Appending lists in logic programming

`append($x , $y , $z)`

means lists `$x` and `$y` append to form `$z`:

`rule(append(null, $y, $y))`

`rule(append(pair($u, $v), $y, pair($u, $z)),
append ($v , $y, $z))`

Appending lists in logic programming

```
rule(append(null, $y, $y))  
rule(append(pair($u, $v), $y, pair($u, $z)),  
      append ($v , $y, $z))
```

```
// Query input:  
append(list("a", "b"), list("c", "d"), $z)
```

```
// Query results:  
append(list("a", "b"), list("c", "d"),  
      list("a", "b", "c", "d"))
```

Using rules backwards

```
rule(append(null, $y, $y))  
rule(append(pair($u, $v), $y, pair($u, $z)),  
      append ($v , $y, $z))
```

```
// Query input:  
append(list("a", "b"), $y , list("a", "b", "c", "d"))
```

```
// Query results:  
append(list("a", "b"), list("c", "d"),  
      list("a", "b", "c", "d"))
```

How many ways to make a given list?

```
rule(append(null, $y, $y))  
rule(append(pair($u, $v), $y, pair($u, $z)),  
      append ($v , $y, $z))
```

// Query input:

```
append($x, $y , list("a", "b", "c", "d"))
```

// Query results :

```
append(null, list("a","b","c","d"), list("a","b","c","d"))  
append(list("a"), list("b","c","d"), list("a","b","c","d"))  
append(list("a","b"), list("c","d"), list("a","b","c","d"))  
append(list("a","b","c"), list("d"), list("a","b","c","d"))  
append(list("a","b","c","d"), null, list("a","b","c","d"))
```

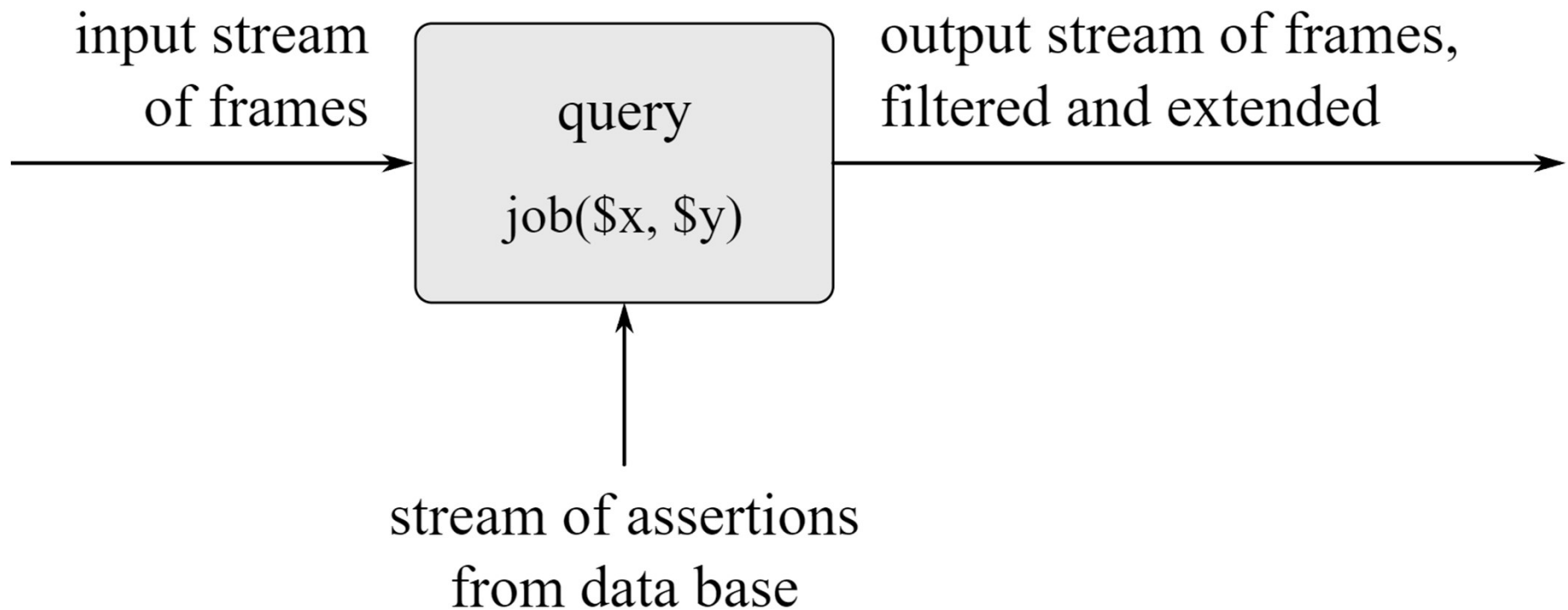
Frames

Frame keeps track of the bindings of logic variables

Example frame

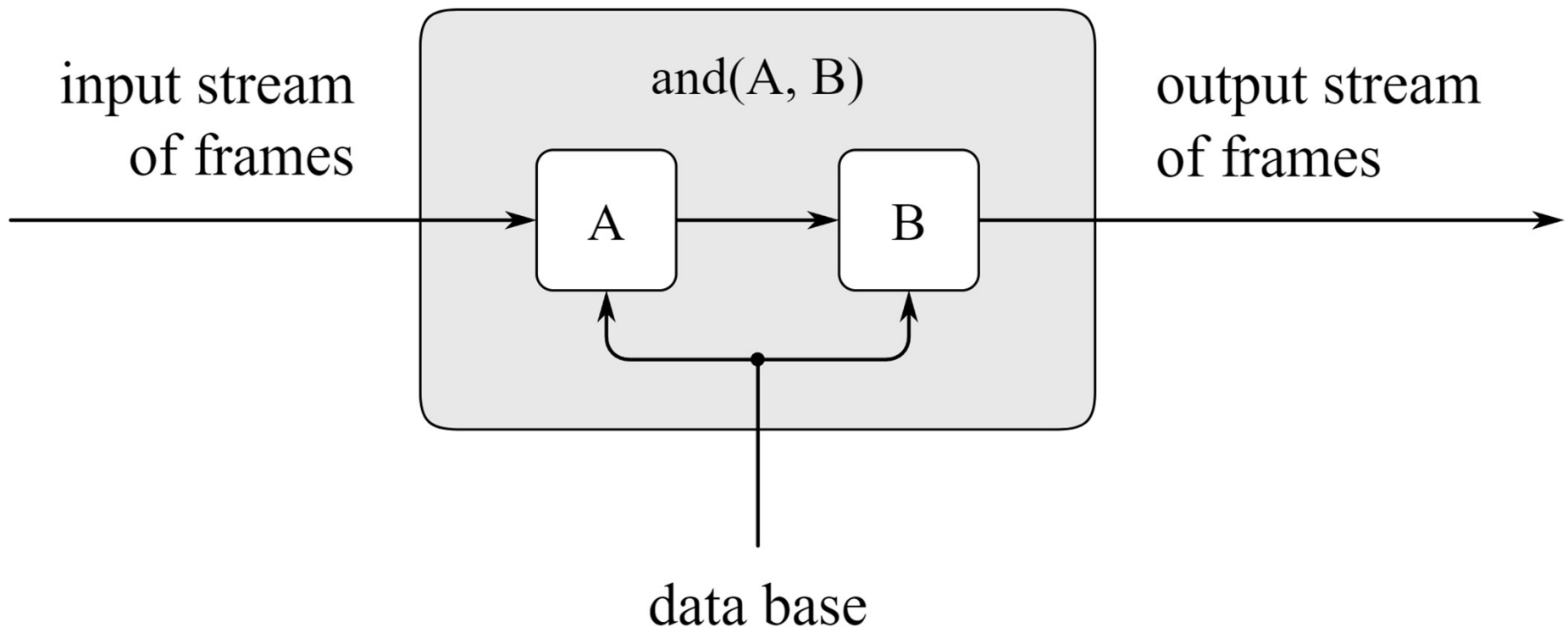
```
$x : ["Reasoner", ["Louis", null]]  
$type : ["programmer" , ["trainee" , null]]
```

Streams of frames



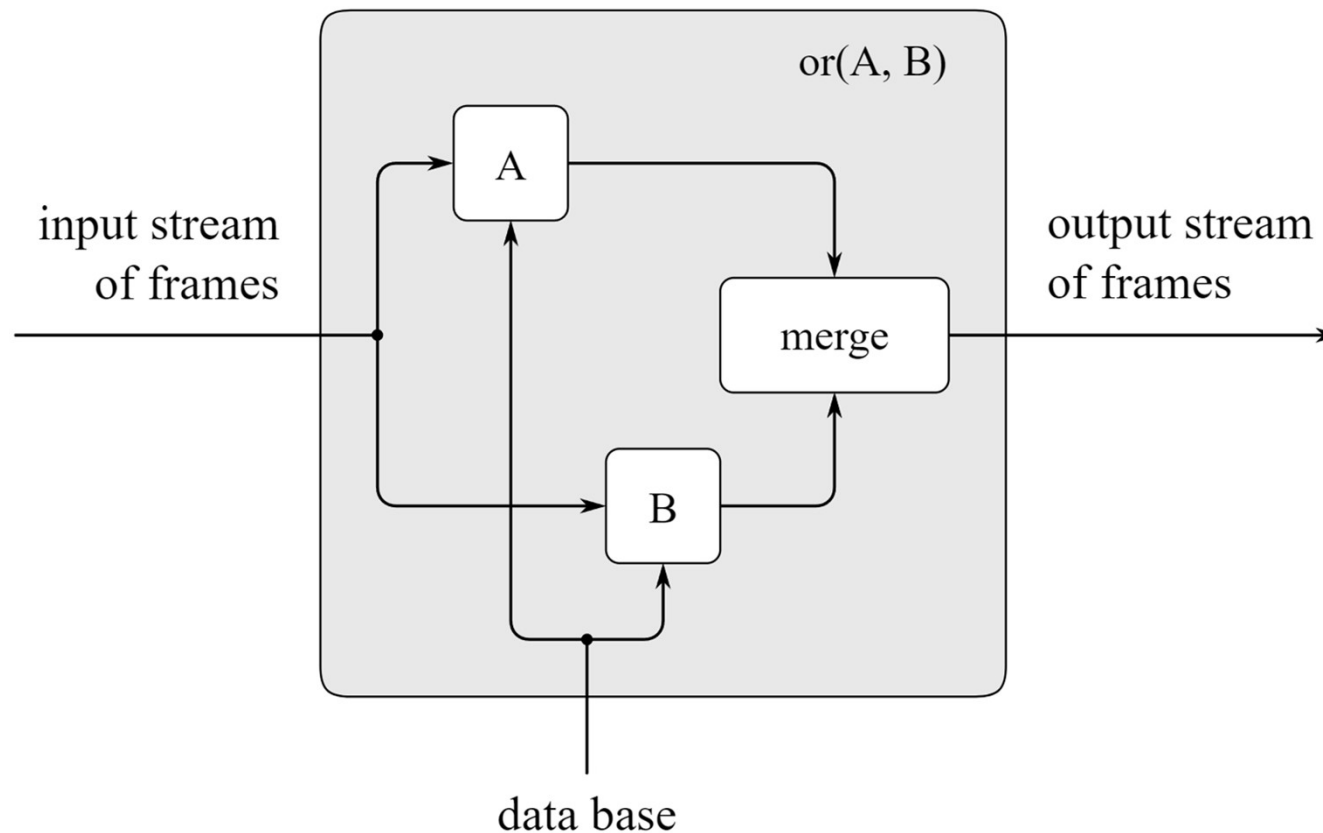
and queries

```
and(can_do_job($x, list("computer", "programmer", "trainee")),  
    job($person , $x))
```



or queries

```
or(supervisor($x, list("Bitdiddle", "Ben")),  
   supervisor($x, list("Hacker", "Alyssa", "P")))
```



Handling rules: unification

- Handle rules in query processing...
 - ... find the rules whose conclusions match a given query pattern
- The job of a unifier for two patterns...
 - ... is to find a frame that makes the patterns equal

Unification example

Unify `list($x, $x)` and

`list(list("a", $y, "c"), list("a", "b", $z))`

`$x = list("a", $y, "c"), $x = list("a", "b", $z)`

`list("a", $y, "c") = list("a", "b", $z )`

`"a" = "a", $y = "b", "c" = $z`

`$x = list("a", "b", "c")`

Another example

Unify `list($x, "a")` and `list(list("b", $y), $z)`

`$x = list("b", $y), "a" = $z`

Remaining binding: `$x` to pattern `list("b", $y)`

Applying rules

- Applying **rules** in **logic programming**
 - Unify the query with the conclusion of the rule to form, if successful, an extension of the original frame.
 - Relative to the extended frame, evaluate the query formed by the body of the rule.
- Applying **functions** in **conventional programming**
 - Bind the function's parameters to its arguments to form a frame that extends the original function environment.
 - Relative to the extended environment, evaluate the expression formed by the body of the function.

Outline

- Some history
- Deductive information retrieval
- How the query system works
- Implementing the query system

Implementing the query system

- Logic Program Evaluator

Summary

- Assertions represent information in a data base
- Queries allow us to retrieve information
- Logic variables let us form patterns
- Compound queries let us form queries from queries
- Rules let us describe (possibly infinite) collections of assertions
- We can *program* with this in interesting ways!
- Query system is an interesting use case for stream processing